56262

Kolmogorov-Arnold Networks 3

Deep Learning Alternatives of the Kolmogorov Superposition Theorem
ICLR (2025)

KAN

- **Basic Idea**
  - High-dimensional functions $y = f(x)$ can be decomposed into compositions of 1D functions

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

$$\phi(x) = w_b \frac{x}{1+e^{-x}} + \sum_{i=0}^{G+k-1} c_i B_i(x)$$

  - These 1D functions are learnable (e.g., spline)

- **Advantages**
  - Interpretability : Each function can be inspected
  - Accuracy : Better approximation of structured (PDE) functions
  - Better scaling laws : Faster improvement with model size

Limitation of KAN

- **Computational Inefficiency**
  - KAN uses spline functions (k), many learnable parameters per edge, grid-based representations (g)
- **Original KST (Kolmogorov Superposition Theorem) is not practical**
  - The original theorem requires many intermediate functions, has complicated constructions, is not directly suitable for modern deep learning.

KST is a powerful theoretical foundation for neural networks, but existing KST-based models (like KAN) are inefficient. Therefore, we need a more practical KST-inspired architecture, which leads to **ActNet**.

Basic Idea of ActNet

- **New KST-inspired Architecture**
  - We introduce ActNet and ActLayer.
- **Theoretical Guarantees**
  - We prove universal approximation properties
- **Better PDE performance**
  - We demonstrate strong performance on: Poisson equations, Helmholtz equations, Allen–Cahn equations
- **Improved efficiency**
  - Compared with KAN: fewer parameters, lower computational cost, better scalability.

KST is Not just One Theorem

$$f(x_1, \ldots, x_d) = \sum_{q=0}^{2d} \Phi_q \left(\sum_{p=1}^{d} \phi_{q,p}(x_p) \right)$$

Outer functions

Inner functions

Version	Formula	Inner Functions	Outer Functions	Other Parameters
Kolmogorov (1957)	$\sum_{q=0}^{2d} \Phi_q \left(\sum_{p=1}^{d} \phi_{q,p}(x_p) \right)$	$2d^2$	$2d$	N/A
Lorentz (1962)	$\sum_{q=0}^{2d} g \left(\sum_{p=1}^{d} \lambda_p \phi_q(x_p) \right)$	$2d$	1	$\lambda \in \mathbb{R}^d$
Sprecher (1965)	$\sum_{q=0}^{2d} g_q \left(\sum_{p=1}^{d} \lambda_p \phi(x_p + qa) \right)$	1	$2d$	$a \in \mathbb{R},\ \lambda \in \mathbb{R}^d$
Laczkovich (2021)	$\sum_{q=1}^{m} g \left(\sum_{p=1}^{d} \lambda_{pq} \phi_q(x_p) \right)$	$m = \mathcal{O}(d)$	1	$\lambda \in \mathbb{R}^{d \times m}$

Simplifies inner functions

Uses shifted functions

More scalable modern formulation

Different KST formulations imply different neural architectures.

Simple Intuition

- **A more scalable alternative based on the theorem proposed by Laczkovich (2021)**

$$f(x) = \sum_{i=1}^{m} g\left(\sum_{j=1}^{d} \lambda_{ij}\phi_i(x_j)\right) \Rightarrow \boxed{f(\boldsymbol{x}) = \sum_{i=1}^{m} g\left(\boldsymbol{\lambda}_i \cdot \phi_i(\boldsymbol{x})\right)}$$

- $f(x)$ is a multivariate target function,
- g is an outer function,
- ϕ_i are inner functions,
- λ_{ij} are coefficients,
- and m controls the representation width.

The d-dimensional input is processed
through $\boldsymbol{m}$ **different functional heads in a multi-head manner.**

Matrix Interpretation

- **The theorem is rewritten in matrix form as:**

$$f(\boldsymbol{x}) = \sum_{i=1}^{m} g\left(\boldsymbol{\lambda}_i \cdot \phi_i(\boldsymbol{x})\right) \quad \Rightarrow \quad f(x) = S\left(g\left[S(\Lambda \odot \Phi(x))\right]\right)$$

- $\Phi(x)$ is the activation matrix,
- Λ is a learnable weight matrix,
- $\odot$ denotes element-wise multiplication,
- and S represents row summation.

ActNet Architecture 1/2

- **ActLayer** (Computational Block)

$$f(x) = S\left(g\left[S(\Lambda \odot \Phi(x))\right]\right)$$

ActLayer

$$\boldsymbol{\Phi}(\boldsymbol{x}) = (\phi_1(\boldsymbol{x}), \ldots, \phi_m(\boldsymbol{x})) = \beta \boldsymbol{B}(\boldsymbol{x})$$

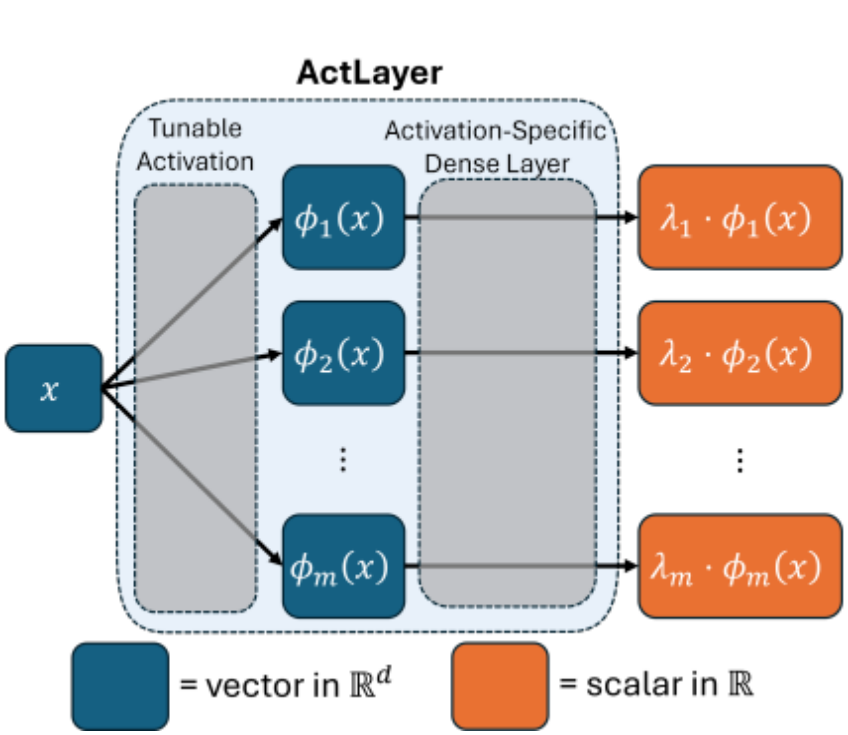
$$\phi_k(t) = \sum_{j=1}^{N} \beta_{kj} b_j(t)$$

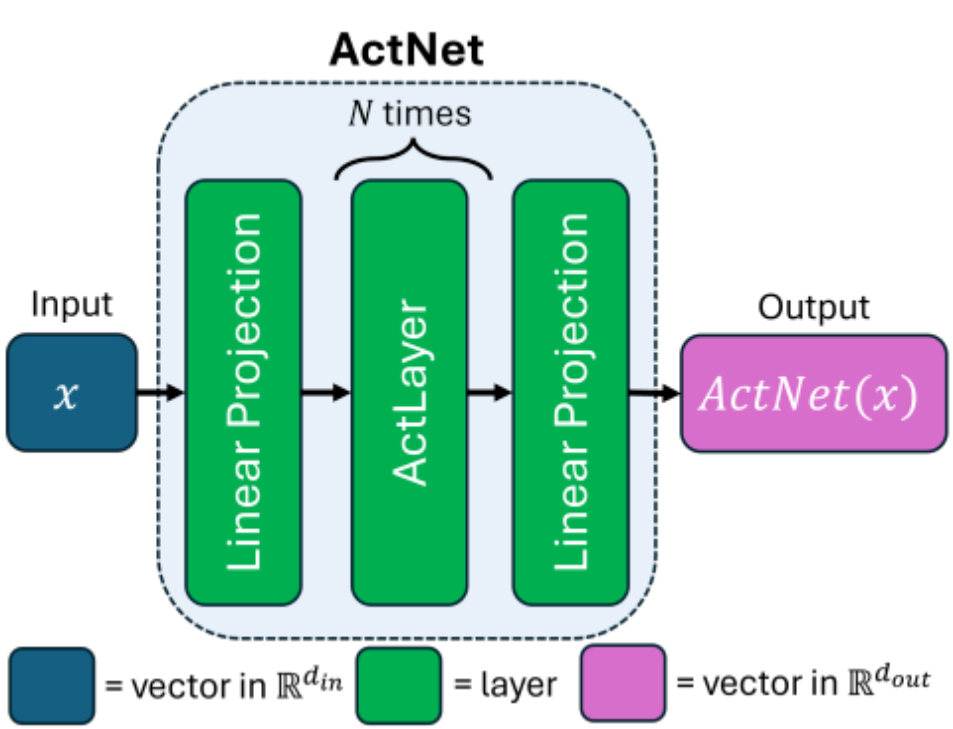
$$\mathrm{ActLayer}_{\beta,\Lambda}(x) = S(\Lambda \odot \beta B(x))$$

ActLayer can be seen as a **Multi-Head MLP** Layer
where each head has a different, tunable, activation function ϕ_i

ActNet Architecture 2/2

$$\text{Input} \rightarrow \text{Linear Projection} \rightarrow \text{ActLayer} \times L \rightarrow \text{Linear Projection} \rightarrow \text{Output}$$





KAN vs. ActNet

	KAN	ActNet
Core Function	Spline-based activation functions	Shared basis-function activations
Activation	Edge-wise spline learning	Multi-head learnable activations
Complexity	$O(Ld^2(G+K))$	$O(Lm(m+N))$

Universal Approximation Property

- A sufficiently wide ActNet with two ActLayers can approximate any continuous function on compact domains.

$$\sup_{x \in \Omega} |f(x) - f_\theta(x)| < \varepsilon.$$

ActNet is theoretially capable of approximating arbitrary continuous multivariate functions because its learnable basis-expanded activation functions can approximate arbitrary one-dimensional functions, which are combined through the superposition structure.

Basis Functions

- **Sinusoidal basis functions** are especially effective for scientific machine learning and PDE problems

$$\phi_k(t) = \sum_{j=1}^{N} \beta_{kj} b_j(t)$$

where:

- $b_j(t)$ are basis functions,
- β_{kj} are learnable coefficients.

$$b_i(t) = \sin(\omega_i t + p_i)$$

- ω_i = frequency,
- p_i = phase shift.

- **Fourier Representation Ability**: Sine functions can represent periodic patterns, oscillatory solutions, high-frequency signals
- **Smooth Higher-Order Derivatives:** sine functions are infinitely differentiable, smooth everywhere
- **Reduced Spectral Bias :** Sinusoidal bases help the network represent high-frequency information earlier and more accurately.

Experiments

	Relative L2 Error (↓)				Residual Loss (↓)			
Benchmark	**ActNet**	**KAN**	**Siren**	**MLP** (base)	**ActNet**	**KAN**	**Siren**	**MLP** (base)
Poisson ($w = 16$)	**6.3e-4**	1.8e-1	9.6e-2	6.6e-2	**4.6e-3**	2.6e0	8.9e-1	4.8e0
Poisson ($w = 32$)	**6.3e-2**	1.1e0	1.8e-1	1.0e0	**8.0e-1**	1.4e+3	8.4e+1	6.2e+4
Helmholtz ($w = 16$)	**1.3e-3**	2.8e-1	1.2e-1	4.6e-2	**5.0e-3**	2.6e0	9.2e-1	6.1e0
Helmholtz ($w = 32$)	**1.1e-1**	1.1e0	1.3e-1	1.0e0	**5.7e-1**	1.5e+3	8.1e+1	6.1e+4
Allen-Cahn	5.6e-5	5.3e-4	**2.1e-5**	1.1e-4	**1.2e-8**	3.8e-8	1.9e-8	2.0e-8

Table 3: Relative L2 errors of ActNet versus state-of-the-art results for PINNs.

Benchmark	**ActNet**	**JaxPi** (Wang et al., 2023)	PirateNet (Wang et al., 2024a)
Advection ($c = 80$)	**9.50e-5**	6.88e-4	5.48e-4
Kuramoto–Sivashinsky (first time window)	1.34e-5	1.42e-4	**1.23e-5**
Kuramoto–Sivashinsky (full solution)	**8.53e-2**	1.61e-1	2.11e-1

Experiments

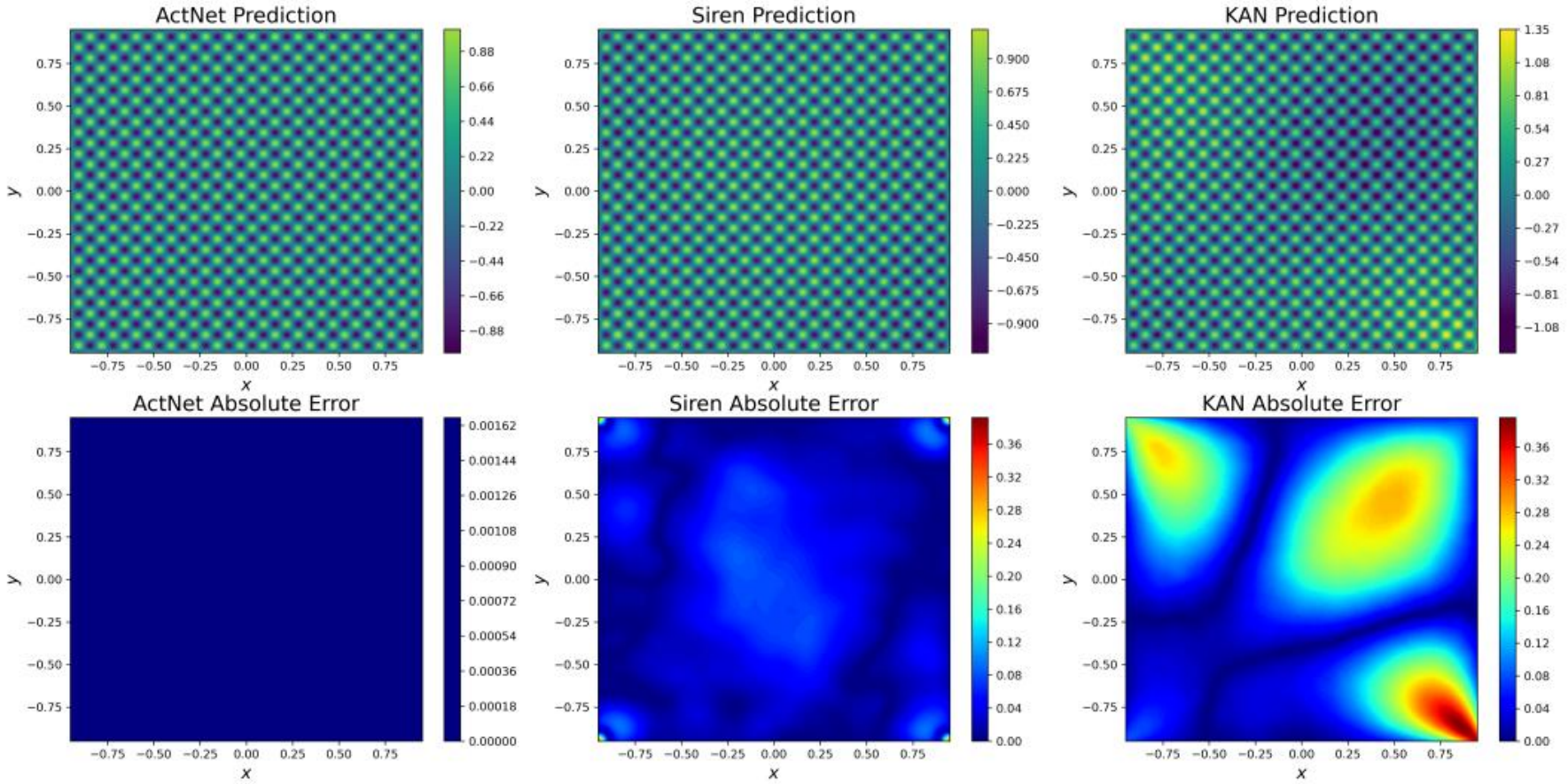


Figure 2: Example predictions for the Helmholtz equation using $w = 16$. The relative L2 errors for the ActNet, Siren and KAN solutions above are 1.04e-03, 8.82e-2 and 2.64e-1, respectively.

Experiments

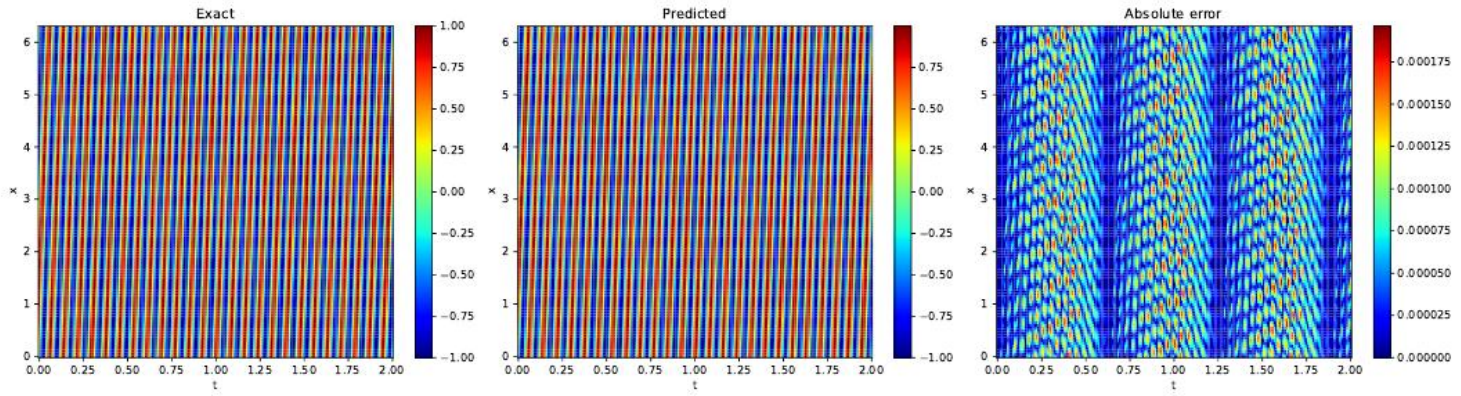


Figure 3: ActNet predictions for the advection equation ($c = 80$). The relative L2 error is 9.50e-5, whereas the best result found in the literature is 6.88e-4 (Wang et al., 2023).

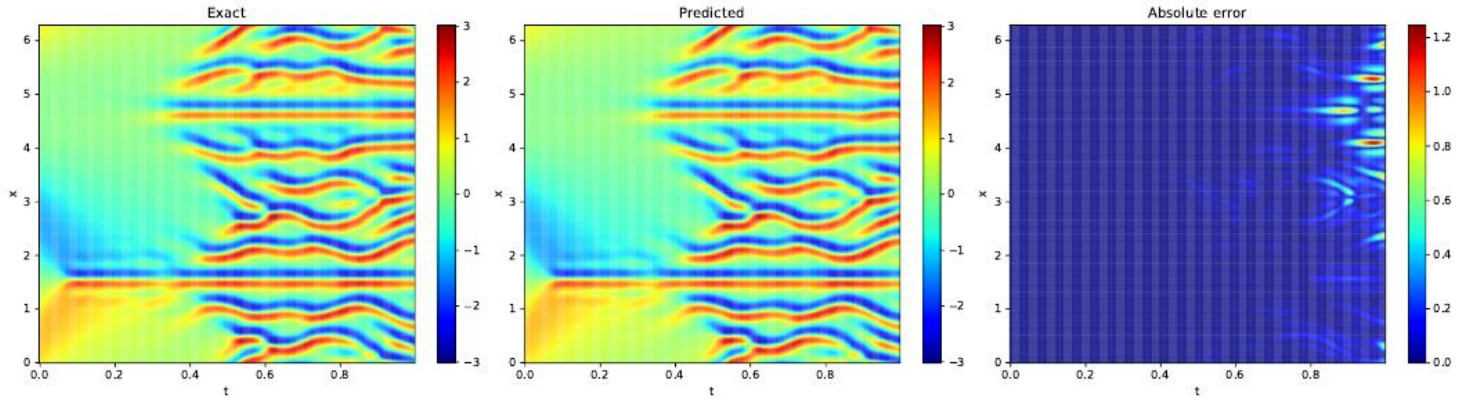


Figure 4: ActNet predictions for the chaotic Kuramoto–Sivashinsky PDE. The relative L2 error is 8.53e-2, whereas the best result found in the literature is 1.61e-1 (Wang et al., 2023).